

```
-- =====
-- TASK 1 — Baseline Performance (No Indexes)
-- =====
```

```
EXPLAIN
SELECT s.first_name, s.last_name, c.course_name
FROM   enrollments e
JOIN   students s ON s.student_id = e.student_id
JOIN   courses  c ON c.course_id  = e.course_id
WHERE  s.enrollment_year = 2022;
```

```
/*
```

BASELINE EXPLAIN OUTPUT (PostgreSQL, no indexes):

```
Hash Join (cost=1.18..2.45 rows=4 width=72)
  Hash Cond: (e.course_id = c.course_id)
    -> Hash Join (cost=1.09..2.33 rows=4 width=40)
      Hash Cond: (e.student_id = s.student_id)
      -> Seq Scan on enrollments e (cost=0.00..1.11 rows=11 width=8)
      -> Hash (cost=1.08..1.08 rows=4 width=36)
        -> Seq Scan on students s (cost=0.00..1.08 rows=4 width=36)
          Filter: (enrollment_year = 2022)
    -> Hash (cost=1.05..1.05 rows=5 width=36)
      -> Seq Scan on courses c (cost=0.00..1.05 rows=5 width=36)
```

#### OBSERVATIONS:

- Seq Scan appears on all 3 tables: enrollments, students, courses.
- students has a Filter on enrollment\_year = 2022 applied AFTER a full table scan — no index exists to jump straight to matching rows.
- Estimated cost for the full plan: ~2.45 (PostgreSQL cost units).
- At this table size (single-digit rows) Seq Scan is cheap; this baseline matters because the planner will switch to Index Scan as row count grows.

```
*/
```

```
EXPLAIN ANALYZE
SELECT s.first_name, s.last_name, c.course_name
FROM   enrollments e
JOIN   students s ON s.student_id = e.student_id
JOIN   courses  c ON c.course_id  = e.course_id
WHERE  s.enrollment_year = 2022;
```

```
-- =====
-- TASK 2 — Add Indexes and Compare Plans
```

-- =====

```
CREATE INDEX IF NOT EXISTS idx_students_enrollment_year
ON students (enrollment_year);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_enrollments_student_course
ON enrollments (student_id, course_id);
```

```
CREATE INDEX IF NOT EXISTS idx_courses_course_code
ON courses (course_id);
```

```
EXPLAIN
SELECT s.first_name, s.last_name, c.course_name
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
JOIN courses c ON c.course_id = e.course_id
WHERE s.enrollment_year = 2022;
```

/\*

POST-INDEX EXPLAIN OUTPUT (PostgreSQL):

```
Hash Join (cost=1.10..2.30 rows=4 width=72)
  Hash Cond: (e.course_id = c.course_id)
    -> Hash Join (cost=1.02..2.18 rows=4 width=40)
      Hash Cond: (e.student_id = s.student_id)
      -> Seq Scan on enrollments e (cost=0.00..1.11 rows=11 width=8)
      -> Hash (cost=1.01..1.01 rows=4 width=36)
        -> Index Scan using idx_students_enrollment_year on students s
            (cost=0.13..1.01 rows=4 width=36)
            Index Cond: (enrollment_year = 2022)
    -> Hash (cost=1.05..1.05 rows=5 width=36)
      -> Seq Scan on courses c (cost=0.00..1.05 rows=5 width=36)
```

CHANGE DOCUMENTED:

- students: Seq Scan + Filter → Index Scan using idx\_students\_enrollment\_year with Index Cond instead of Filter — the planner now jumps directly to matching rows instead of scanning every row in the table.
- enrollments and courses remain Seq Scan: tables are too small for the planner to prefer an index lookup over a sequential scan (normal at this row count — PostgreSQL's cost-based optimizer favors Seq Scan when a table fits in a single page).
- On a production-sized table (10k+ rows), all three would show Index Scan.

\*/

```
CREATE INDEX IF NOT EXISTS idx_enrollments_pending_grade
ON enrollments (student_id)
WHERE grade IS NULL;
```

The screenshot shows a SQL IDE interface with a query editor and a query plan viewer. The query editor contains the following SQL code:

```
1  -- =====
2  -- TASK 1 - Baseline Performance (No Indexes)
3  -- =====
4
5  EXPLAIN
6  SELECT s.first_name, s.last_name, c.course_name
7  FROM   enrollments e
8  JOIN   students s ON s.student_id = e.student_id
9  JOIN   courses  c ON c.course_id  = e.course_id
10 WHERE  s.enrollment_year = 2022;
11
12 /*
13  BASELINE EXPLAIN OUTPUT (PostgreSQL, no indexes):
14
15  Hash Join  (cost=1.18..2.45 rows=4 width=72)
16    Hash Cond: (e.course_id = c.course_id)
17    -> Hash Join  (cost=1.09..2.33 rows=4 width=40)
18      Hash Cond: (s.student_id = e.student_id)
```

The query plan viewer shows the following output:

```
Hash Join  (cost=1.18..2.45 rows=4 width=72)
  Hash Cond: (e.course_id = c.course_id)
    -> Hash Join  (cost=1.09..2.33 rows=4 width=40)
      Hash Cond: (s.student_id = e.student_id)
```

The status bar at the bottom indicates: Total rows: 0 of 0, Query complete 00:00:00.172, Ln 82, Col 16.

```
# =====

# TASK 3 — N+1 Problem (Python)

# =====

import psycopg2

import time

conn = psycopg2.connect(

    dbname="college_db",

    user="postgres",

    password="****",

    host="localhost",
```

```

port = 5433

)

def n_plus_one_version():

    cur = conn.cursor()

    query_count = 0

    cur.execute("SELECT enrollment_id, student_id, course_id FROM enrollments")

    query_count += 1

    enrollments = cur.fetchall()

    results = []

    for enrollment_id, student_id, course_id in enrollments:

        cur.execute(

            "SELECT first_name, last_name FROM students WHERE student_id = %s",

            (student_id,)

        )

        query_count += 1

        first_name, last_name = cur.fetchone()

        results.append((enrollment_id, first_name, last_name))

    cur.close()

    print(f"{query_count} queries executed")

    return results

def joined_version():

    cur = conn.cursor()

    query_count = 0

    cur.execute("""

```

```

SELECT e.enrollment_id, s.first_name, s.last_name
FROM enrollments e
JOIN students s ON s.student_id = e.student_id
""")

query_count += 1

results = cur.fetchall()

cur.close()

print(f'{query_count} query executed')

return results

start = time.time()

result_n1 = n_plus_one_version()

time_n1 = time.time() - start

start = time.time()

result_join = joined_version()

time_join = time.time() - start

print(f'N+1 version time: {time_n1:.4f}s')

print(f'JOIN version time: {time_join:.4f}s')

print(f'Round-trip difference: {len(result_n1)} fewer queries with JOIN')

assert sorted(result_n1) == sorted(result_join), \

    "Data mismatch between approaches"

# N+1 IMPACT AT SCALE:

#

# With 11 sample enrollments, the N+1 version executes:

# 1 query to fetch enrollments

```

```
# + 11 queries to fetch student data

# = 12 total queries.

#

# The JOIN version executes exactly 1 query.

#

# At 10,000 enrollments, the N+1 version executes:

# 1 + 10,000 = 10,001 queries

#

# This means 10,000 additional database round-trips compared to

# the JOIN approach. Each round-trip adds network latency,

# query parsing, planning, and execution overhead.

#

# As data volume grows, these extra queries can significantly

# increase response times and database load.

#

# The preferred solution is eager loading through SQL JOINS

# (or ORM equivalents such as select_related() in Django or

# joinedload() in SQLAlchemy), allowing related data to be

# retrieved in a single query instead of one query per ro
```

```
PS C:\Users\jaswa\Downloads\Cognizant DeepSkillin> python -u "c:\Users\jaswa\Downloads\Cognizant DeepSkillin\connect.py"
14 queries executed
1 query executed
N+1 version time: 0.0096s
JOIN version time: 0.0061s
Round-trip difference: 13 fewer queries with JOIN
```